

PODATKOVNE BAZE I

Informatika in podatkovne tehnologije (IPT), Telekomunikacije (TK)
Univerzitetni študijski program (UN)

1. letnik

Preden začnemo...

- Ustvarite tabelo ŠKROPIVO.
 - Ustvarite tabelo TIP_ŠKROPIVA.
 - Povezave!
-
- **+10% na 2. kolokviju** (če je ocena pozitivna).

Preden začnemo...

- Kako izpišemo **vse** podatke iz tabele?
- Kako izpišemo podatke samo **določenih** stolpcev?
- Kako izpišemo vse **različne** podatke iz tabele?
- Kako preimenujemo stolpce pri **izpisu**?
- Kako poizvedujemo glede na **pogoj**?
 - Osnovni primerjali predikati.
- Kako poizvedujemo glede na **več pogojev**?

DQL

Data Query Language

(nadaljevanje)

Ujemanje znakovnih nizov

- Uporaba predikata **LIKE**.
 - Iskanje specifičnega vzorca znakovnih nizov.
 - _ - nadomeščanje natanko enega znaka
 - % - nadomeščanje poljubnih znakov
- *Izpišite vse nazive rastlin, katerih tip rastline vsebuje ključno besedo „rastlina“.*
- *Izpišite vse nazive rastlin, katerih tip rastline ne vsebuje ključne besede „rastlina“.*
- *Izpišite imena strank, ki se začnejo na T.*

<https://www.postgresql.org/docs/18/functions-matching.html>

IN

- Predikat **IN** uporabljamo za preverjanje obstoja določene celice tabele v definirani množici. Lahko ga ponazorimo s simbolom **€**.
- Posredno vključuje osnovni predikat enakosti, z razliko, da se **enakost ne preverja nad eno vrednostjo**, ampak **nad številnimi**.
- Vrača vrednost **PRAVILNO**, če je enakost **vsaj enkrat zadoščena** na podano množico.
- Lahko si razlagamo tudi s kombinacijo operatorja **OR** (||) in predikata enakosti nad številnimi vrednostmi:
 - `iskana_vrednost = 1 OR iskana_vrednost = 2 OR iskana_vrednost = 3 OR iskana_vrednost = 4`
 - `iskana_vrednost IN (1, 2, 3, 4)`

NOT IN

- Predikat **NOT IN** vrača pravilno, če enakost **ni zadoščena**.
- Seznam vrednosti je lahko **neposredno** definiran ali pa je definiran s pomočjo drugega (**vgnezdenega**) **SELECT stavka**.
 - Vgnezen stavek mora vračati vrednosti samo za **en stolpec**.
 - Vgnezeni stavki se izvedejo **prvi**!
 - Na podoben način, z uporabo predikata enakosti (=), se ne more poizvedovati, saj predikat enakosti zahteva eno vrednost, vgnezen stavek pa lahko vrne več vrednosti.

IN

- *Izpišite vse nazive rastlin, katerih cena je 2 ali 4 EUR.*
- *Izpišite vse podatke računov, ki jih je kupila oseba z imenom Tina.*
- *Izpišite nazive rastlin, katerih cena ni 2 ali 4 EUR.*
- *Izpišite imena in priimke strank, ki niso stalne stranke.*

EXISTS

- Predikat **EXISTS** je podoben predikatu **IN**, vendar je za njegovo delovanje **nujna opredelitev vgnezdenega stavka** (pri predikatu IN lahko seznam vrednosti definiramo ročno).
- Predikat vrača **PRAVILNO**, ko poizvedba vrne vsaj en rezultat (kardinalnost podpoizvedbe večja od 0) in **NAPAČNO**, če je **rezultat podpoizvedbe prazen**.
 - Pri IN primerjamo, pri EXISTS štejemo moč množice.
- V podpoizvedbi lahko **izberemo vse** ali **posamezne stolpce**.
- Predikat EXISTS ustavi izvajanje podpoizvedbe takoj, ko najde prvo ujemanje oz. prvi pravilen rezultat glede na samo poizvedbo.

EXISTS

- *Izpišite ime in priimek stranke, ki ima vsaj en račun.*
 - Za vsako vrstico v tabeli STRANKA poizvedba preveri id stranke (tk_stranka) v tabeli RAČUN. Če se id_stranke, ki se pojavi v tabeli STRANKA, obstaja tudi v tabeli RAČUN, podpoizvedbo vrne prvo ujemajočo vrstico. Kot rezultat predikat EXISTS vrne PRAVILNO in preneha pregledovati tabelo RAČUN. V nasprotnem primeru podpoizvedbo ne vrne nobene vrstice in predikat EXISTS vrne NAPAČNO.
 - Uporaba, v primerjavi z uporabo običajne povezave med dvema tabelama je vsekakor HITROST, saj EXISTS prekine podpoizvedbo takoj, ko najde ujemajoči vrstici.

Funkcije nad množicami

Funkcije nad množicami

- Tiste funkcije, ki **procesirajo podatke** iz tabel in kot rezultat vrnejo vrednost, na primer število vrstic v tabeli, **minimalno**, **maksimalno** ali **povprečno** vrednost, **vsoto** vrednosti in drugo.
- Funkcije nad množicami imenujemo tudi **agregatne funkcije**, ker kot **rezultat** vrnejo **agregatno** (skupno) **vrednost**.

<https://www.postgresql.org/docs/18/functions-aggregate.html>

Funkcije nad množicami

Funkcija	Ime	Opis
COUNT()	Štetje	Vrne število vseh zapisov določenega stolpca tabele.
SUM()	Vsota	Vrne vsoto vseh vrednosti določenega stolpca tabele.
AVG()	Povprečje	Vrne povprečno vrednost vseh vrednosti določenega stolpca tabele.
MIN()	Minimum	Vrne najmanjšo vrednost določenega stolpca tabele.
MAX()	Maksimum	Vrne največjo vrednost določenega stolpca tabele.

Štetje

- Funkcija **COUNT** vrne **število zapisov** v skupini (tabeli).
- Uporabimo jo lahko na način:
 - **COUNT(*)** – preštevava vse zapise določene tabele
 - **COUNT(stolpec)** – preštevava vse ne prazne (NOT NULL) vrednosti podanega stolpca
 - **COUNT(DISTINCT stolpec)** – preštevava samo unikatne zapise

Štetje

- *Koliko strank ima naša vrtnarija?*
- *Koliko podjetji je med našimi strankami?*
- *Iz koliko različnih krajev prihajajo naše stranke?*
- *Koliko našim strankam je ime Tina?*

Vsota in povprečje

- Funkciji **SUM** in **AVG** lahko uporabimo le nad **numeričnimi** podatkovnimi tipi (npr. INTEGER, DECIMAL, DOUBLE, FLOAT).
- Ne moremo uporabiti *, saj delamo nad številskimi vrednostmi.
- Tudi tukaj ključna beseda **DISTINCT** povzroči upoštevanje zgolj unikatnih vrednosti.
- Uporabimo lahko več funkcij hkrati.

Vsota in povprečje

- *Kolikšna je skupna vrednost vseh izdanih računov v naši vrtnariji?*
- *Kolikšna je skupna vrednost vseh izdanih računov v naši vrtnariji v letu 2023?*
- *Kolikšna je vsota zgolj unikatnih vrednosti izdanih računov?*
- *Kolikšna je povprečna vrednost vseh izdanih računov v letu 2022?*

Lahko tudi – standardni odklon

- Funkcija **STDDEV** vrne standardni deviacijo vseh vrednosti v stolpcu.
- Primer:
 - *Kakšna je standardna deviacija vseh računov naše vrtnarije?*

Minimum in maksimum

- Funkciji **MIN** in **MAX** nista omejeni zgolj na numerične podatkovne tipe. Uporaba nad znakovnimi nizi vrne najdaljši oz. najkrajši znakovni zapis.
- Ne moremo uporabiti *, nujno je, da definiramo stolpec.
- Uporabimo lahko več funkcij hkrati.

Minimum in maksimum

- *Koliko stane najdražji artikel?*
 - *Koliko stane najcenejši artikel med cvetočimi rastlinami?*
 - *Kakšni sta najvišja in najnižja vrednost izdanih računov?*
-
- *Pazljivo... Naj nas ne zanese – Kateri artikel je najdražji?*

Izbira funkcij in stolpcev hkrati

- Primer - Kateri artikel je najdražji?
 - Stavek, če bi bil veljaven, bi vrnil za vsak artikel maksimalno ceno, ki pa je samo ena. Veljaven stavek za to je naveden spodaj, ni pa smiseln, saj še vedno ne vemo kateri artikel je najdražji.
 - Funkcije nad množicami (npr. COUNT) uporabljamo v SELECT delu ukaza SQL. Njihova hkratna uporaba z izbiro stolpcev tabele je sintaktično pravilna, vendar nesmiselna – če je uporabljena brez GROUP BY.
 - Rešujemo po korakih:
 - 1. Najdemo ceno, ki je najvišja
 - 2. Najdeno ceno primerjamo s ceno vsakega posameznega artikla
 - 3. Izpišemo (**VSE!**) artikle, kjer se ceni ujemata